



API Liga Deportiva

Sports League API

Fase 4: Match y Programación de Partidos

Conceptos Clave

Múltiples FK a la misma tabla | DeleteBehavior.Restrict
Ciclos de Cascada | Validaciones Complejas de Negocio

Guía paso a paso para construcción en clase

1. Objetivos de la Fase 4

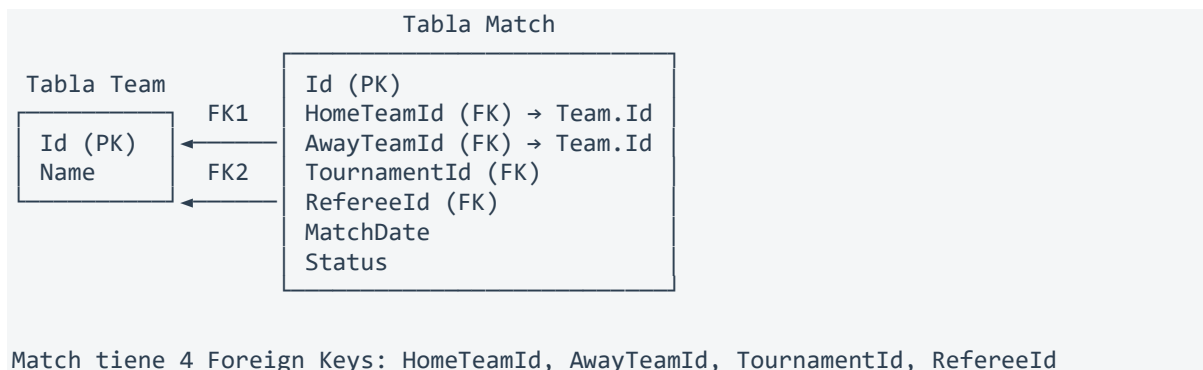
Al finalizar esta fase, los estudiantes serán capaces de:

- Crear entidades con múltiples Foreign Keys que apuntan a la misma tabla
- Entender y resolver el problema de ciclos de cascada con DeleteBehavior.Restrict
- Configurar Navigation Properties múltiples hacia la misma entidad
- Implementar validaciones de negocio complejas que cruzan múltiples entidades
- Usar la máquina de estados para controlar el ciclo de vida de un partido
- Actualizar Navigation Properties en entidades existentes (Team, Tournament, Referee)

2. Conceptos Nuevos en esta Fase

2.1 Múltiples Foreign Keys a la misma tabla

La tabla Match tiene dos columnas que apuntan a Team: HomeTeamId (equipo local) y AwayTeamId (equipo visitante). Esto genera dos relaciones 1:N independientes desde Team hacia Match:



2.2 El problema de los ciclos de cascada

Cuando una tabla tiene múltiples FKs hacia la misma tabla con DeleteBehavior.Cascade, SQL Server detecta múltiples caminos de cascada y rechaza la migración con un error:

⚠ Error de SQL Server

Introducing FOREIGN KEY constraint on table 'Matches' may cause cycles or multiple cascade paths. Specify ON DELETE NO ACTION or ON DELETE SET NULL, or modify other FOREIGN KEY constraints.

La solución es usar `DeleteBehavior.Restrict` en las FKs hacia `Team`. Esto significa que no se puede eliminar un `Team` si tiene partidos asociados. El usuario debe eliminar primero los partidos manualmente.

```
// X PROBLEMA: Dos cascadas desde Team hacia Match
// Si elimino Team, ¿qué pasa con Match? ¿Se elimina por HomeTeamId o por AwayTeamId?
// SQL Server no permite ambigüedad en cascadas.

// ✓ SOLUCIÓN: Usar Restrict en ambas FKs hacia Team
.OnDelete>DeleteBehavior.Restrict) // No permite eliminar Team si tiene Matches
```

💡 DeleteBehavior explicado

Cascade: eliminar padre elimina hijos automáticamente. **Restrict:** no permite eliminar padre si tiene hijos (lanza error). **SetNull:** pone la FK en null (solo si es nullable). En nuestro caso, **Restrict** es lo más seguro: no queremos perder partidos accidentalmente al eliminar un equipo.

3. Crear el Enum MatchStatus

SportsLeague.Domain/Enums/MatchStatus.cs

```
namespace SportsLeague.Domain.Enums;

public enum MatchStatus
{
    Scheduled = 0,
    InProgress = 1,
    Finished = 2,
    Suspended = 3
}
```

Las transiciones válidas de `MatchStatus`:



4. Crear la Entity Match

SportsLeague.Domain/Entities/Match.cs

```
using SportsLeague.Domain.Enums;

namespace SportsLeague.Domain.Entities;

public class Match : AuditBase
{
    public int TournamentId { get; set; }
    public int HomeTeamId { get; set; }
    public int AwayTeamId { get; set; }
    public int RefereeId { get; set; }
    public DateTime MatchDate { get; set; }
    public string Venue { get; set; } = string.Empty;
    public int Matchday { get; set; }
    public MatchStatus Status { get; set; } = MatchStatus.Scheduled;

    // Navigation Properties
    public Tournament Tournament { get; set; } = null!;
    public Team HomeTeam { get; set; } = null!;
    public Team AwayTeam { get; set; } = null!;
    public Referee Referee { get; set; } = null!;
}
```

Cuatro Navigation Properties

Match tiene 4 Navigation Properties, una por cada FK. Esto permite acceder a match.HomeTeam.Name, match.AwayTeam.Name, match.Tournament.Name y match.Referee.LastName sin queries adicionales (usando .Include()).

5. Actualizar Entidades Existentes

Agregar las Navigation Properties de colección en las entidades que son referenciadas por Match:

5.1 Actualizar Team

Agregar las dos colecciones de partidos (como local y como visitante):

```
// Agregar dentro de la clase Team, después de TournamentTeams:

// Partidos como local
public ICollection<Match> HomeMatches { get; set; } = new List<Match>();
// Partidos como visitante
```

```
public ICollection<Match> AwayMatches { get; set; } = new List<Match>();
```

5.2 Actualizar Tournament

Agregar la colección de partidos:

```
// Agregar dentro de la clase Tournament, después de TournamentTeams:

public ICollection<Match> Matches { get; set; } = new List<Match>();
```

5.3 Actualizar Referee

Agregar la colección de partidos dirigidos:

```
// Agregar dentro de la clase Referee:

public ICollection<Match> Matches { get; set; } = new List<Match>();
```

6. Actualizar el DbContext

Agregar el DbSet y la configuración de Match con Fluent API:

Agregar el DbSet

```
public DbSet<Match> Matches => Set<Match>();
```

Configuración de Match en OnModelCreating

```
// — Match Configuration —
modelBuilder.Entity<Match>(entity =>
{
    entity.HasKey(m => m.Id);
    entity.Property(m => m.MatchDate)
        .IsRequired();
    entity.Property(m => m.Venue)
        .HasMaxLength(150);
    entity.Property(m => m.Matchday)
        .IsRequired();
    entity.Property(m => m.Status)
        .IsRequired();
    entity.Property(m => m.CreatedAt)
        .IsRequired();
    entity.Property(m => m.UpdatedAt)
        .IsRequired(false);
```

```
// Relación con Tournament (Cascade: eliminar torneo elimina partidos)
entity.HasOne(m => m.Tournament)
    .WithMany(t => t.Matches)
    .HasForeignKey(m => m.TournamentId)
    .OnDelete(DeleteBehavior.Cascade);

// Relación con HomeTeam (Restrict: evita ciclo de cascada)
entity.HasOne(m => m.HomeTeam)
    .WithMany(t => t.HomeMatches)
    .HasForeignKey(m => m.HomeTeamId)
    .OnDelete(DeleteBehavior.Restrict);

// Relación con AwayTeam (Restrict: evita ciclo de cascada)
entity.HasOne(m => m.AwayTeam)
    .WithMany(t => t.AwayMatches)
    .HasForeignKey(m => m.AwayTeamId)
    .OnDelete(DeleteBehavior.Restrict);

// Relación con Referee (Restrict: no eliminar árbitro con partidos)
entity.HasOne(m => m.Referee)
    .WithMany(r => r.Matches)
    .HasForeignKey(m => m.RefereeId)
    .OnDelete(DeleteBehavior.Restrict);
});
```

💡 ¿Por qué Cascade solo en Tournament?

Si eliminamos un torneo, tiene sentido eliminar todos sus partidos (Cascade). Pero si eliminamos un equipo o árbitro, NO queremos perder el historial de partidos. Por eso usamos Restrict en Team y Referee: obliga al administrador a reasignar o eliminar partidos primero.

⚠️ Error común

Si se usa Cascade en ambas FKs de Team (HomeTeamId y AwayTeamId), SQL Server lanzará un error de ciclo de cascada al aplicar la migración. SIEMPRE usar Restrict cuando hay múltiples FKs hacia la misma tabla.

7. Definir las Interfaces de Match

7.1 IMatchRepository

SportsLeague.Domain/Interfaces/Repositories/IMatchRepository.cs

```

using SportsLeague.Domain.Entities;

namespace SportsLeague.Domain.Interfaces.Repositories;

public interface IMatchRepository : IGenericRepository<Match>
{
    Task<IEnumerable<Match>> GetByTournamentAsync(int tournamentId);
    Task<IEnumerable<Match>> GetByTeamAsync(int teamId);
    Task<Match?> GetByIdWithDetailsAsync(int id);
    Task<IEnumerable<Match>> GetByTournamentWithDetailsAsync(int tournamentId);
}

```

7.2 IMatchService

SportsLeague.Domain/Interfaces/Services/IMatchService.cs

```

using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Enums;

namespace SportsLeague.Domain.Interfaces.Services;

public interface IMatchService
{
    Task<IEnumerable<Match>> GetAllByTournamentAsync(int tournamentId);
    Task<Match?> GetByIdAsync(int id);
    Task<Match> CreateAsync(Match match);
    Task UpdateAsync(int id, Match match);
    Task DeleteAsync(int id);
    Task UpdateStatusAsync(int id, MatchStatus newStatus);
}

```

8. Implementar el MatchRepository

SportsLeague.DataAccess/Repositories/MatchRepository.cs

```

using Microsoft.EntityFrameworkCore;
using SportsLeague.DataAccess.Context;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Repositories;

namespace SportsLeague.DataAccess.Repositories;

public class MatchRepository : GenericRepository<Match>, IMatchRepository
{
    public MatchRepository(LeagueDbContext context) : base(context)
    {
    }

    public async Task<IEnumerable<Match>> GetByTournamentAsync(int tournamentId)
    {
    }
}

```

```

{
    return await _dbSet
        .Where(m => m.TournamentId == tournamentId)
        .OrderBy(m => m.Matchday)
        .ThenBy(m => m.MatchDate)
        .ToListAsync();
}

public async Task<IEnumerable<Match>> GetByTeamAsync(int teamId)
{
    return await _dbSet
        .Where(m => m.HomeTeamId == teamId || m.AwayTeamId == teamId)
        .Include(m => m.HomeTeam)
        .Include(m => m.AwayTeam)
        .OrderBy(m => m.MatchDate)
        .ToListAsync();
}

public async Task<Match?> GetByIdWithDetailsAsync(int id)
{
    return await _dbSet
        .Include(m => m.Tournament)
        .Include(m => m.HomeTeam)
        .Include(m => m.AwayTeam)
        .Include(m => m.Referee)
        .FirstOrDefaultAsync(m => m.Id == id);
}

public async Task<IEnumerable<Match>> GetByTournamentWithDetailsAsync(
    int tournamentId)
{
    return await _dbSet
        .Where(m => m.TournamentId == tournamentId)
        .Include(m => m.HomeTeam)
        .Include(m => m.AwayTeam)
        .Include(m => m.Referee)
        .OrderBy(m => m.Matchday)
        .ThenBy(m => m.MatchDate)
        .ToListAsync();
}
}

```

💡 Múltiples Include

GetByIdWithDetailsAsync usa 4 .Include() para cargar todas las Navigation Properties en una sola query. EF Core genera un SELECT con 4 JOINS. GetByTeamAsync usa un OR en el Where para buscar partidos donde el equipo es local O visitante.

9. Implementar el MatchService

Este Service tiene las validaciones más complejas del proyecto hasta ahora:

SportsLeague.Domain/Services/MatchService.cs

```
using Microsoft.Extensions.Logging;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Enums;
using SportsLeague.Domain.Interfaces.Repositories;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.Domain.Services;

public class MatchService : IMatchService
{
    private readonly IMatchRepository _matchRepository;
    private readonly ITournamentRepository _tournamentRepository;
    private readonly ITournamentTeamRepository _tournamentTeamRepository;
    private readonly ITeamRepository _teamRepository;
    private readonly IRefereeRepository _refereeRepository;
    private readonly ILogger<MatchService> _logger;

    public MatchService(
        IMatchRepository matchRepository,
        ITournamentRepository tournamentRepository,
        ITournamentTeamRepository tournamentTeamRepository,
        ITeamRepository teamRepository,
        IRefereeRepository refereeRepository,
        ILogger<MatchService> logger)
    {
        _matchRepository = matchRepository;
        _tournamentRepository = tournamentRepository;
        _tournamentTeamRepository = tournamentTeamRepository;
        _teamRepository = teamRepository;
        _refereeRepository = refereeRepository;
        _logger = logger;
    }

    public async Task<IEnumerable<Match>> GetAllByTournamentAsync(int tournamentId)
    {
        var tournament = await _tournamentRepository.GetByIdAsync(tournamentId);
        if (tournament == null)
            throw new KeyNotFoundException(
                $"No se encontró el torneo con ID {tournamentId}");

        return await _matchRepository.GetByTournamentWithDetailsAsync(tournamentId);
    }

    public async Task<Match?> GetByIdAsync(int id)
    {
        _logger.LogInformation("Retrieving match with ID: {MatchId}", id);
        return await _matchRepository.GetByIdWithDetailsAsync(id);
    }
}
```

```
public async Task<Match> CreateAsync(Match match)
{
    // 1. Validar que el torneo existe y está en InProgress
    var tournament = await
_tournamentRepository.GetByIdAsync(match.TournamentId);
    if (tournament == null)
        throw new KeyNotFoundException(
            $"No se encontró el torneo con ID {match.TournamentId}");

    if (tournament.Status != TournamentStatus.InProgress)
        throw new InvalidOperationException(
            "Solo se pueden programar partidos en torneos con estado
InProgress");

    // 2. Validar que los equipos son diferentes
    if (match.HomeTeamId == match.AwayTeamId)
        throw new InvalidOperationException(
            "El equipo local y visitante deben ser diferentes");

    // 3. Validar que ambos equipos existen
    var homeTeamExists = await _teamRepository.ExistsAsync(match.HomeTeamId);
    if (!homeTeamExists)
        throw new KeyNotFoundException(
            $"No se encontró el equipo local con ID {match.HomeTeamId}");

    var awayTeamExists = await _teamRepository.ExistsAsync(match.AwayTeamId);
    if (!awayTeamExists)
        throw new KeyNotFoundException(
            $"No se encontró el equipo visitante con ID {match.AwayTeamId}");

    // 4. Validar que ambos equipos están inscritos en el torneo
    var homeEnrolled = await _tournamentTeamRepository
        .GetByTournamentAndTeamAsync(match.TournamentId, match.HomeTeamId);
    if (homeEnrolled == null)
        throw new InvalidOperationException(
            "El equipo local no está inscrito en este torneo");

    var awayEnrolled = await _tournamentTeamRepository
        .GetByTournamentAndTeamAsync(match.TournamentId, match.AwayTeamId);
    if (awayEnrolled == null)
        throw new InvalidOperationException(
            "El equipo visitante no está inscrito en este torneo");

    // 5. Validar que el árbitro existe
    var refereeExists = await _refereeRepository.ExistsAsync(match.RefereeId);
    if (!refereeExists)
        throw new KeyNotFoundException(
            $"No se encontró el árbitro con ID {match.RefereeId}");

    match.Status = MatchStatus.Scheduled;

    _logger.LogInformation(
        "Creating match: Team {Home} vs Team {Away} in Tournament {Tournament}",
        match.HomeTeamId, match.AwayTeamId, match.TournamentId);
    return await _matchRepository.CreateAsync(match);
}
```

```

    }

    public async Task UpdateAsync(int id, Match match)
    {
        var existing = await _matchRepository.GetByIdAsync(id);
        if (existing == null)
            throw new KeyNotFoundException($"No se encontró el partido con ID {id}");

        if (existing.Status != MatchStatus.Scheduled)
            throw new InvalidOperationException(
                "Solo se pueden editar partidos con estado Scheduled");

        // Mismas validaciones que en Create
        if (match.HomeTeamId == match.AwayTeamId)
            throw new InvalidOperationException(
                "El equipo local y visitante deben ser diferentes");

        var homeEnrolled = await _tournamentTeamRepository
            .GetByTournamentAndTeamAsync(existing.TournamentId, match.HomeTeamId);
        if (homeEnrolled == null)
            throw new InvalidOperationException(
                "El equipo local no está inscrito en este torneo");

        var awayEnrolled = await _tournamentTeamRepository
            .GetByTournamentAndTeamAsync(existing.TournamentId, match.AwayTeamId);
        if (awayEnrolled == null)
            throw new InvalidOperationException(
                "El equipo visitante no está inscrito en este torneo");

        var refereeExists = await _refereeRepository.ExistsAsync(match.RefereeId);
        if (!refereeExists)
            throw new KeyNotFoundException(
                $"No se encontró el árbitro con ID {match.RefereeId}");

        existing.HomeTeamId = match.HomeTeamId;
        existing.AwayTeamId = match.AwayTeamId;
        existing.RefereeId = match.RefereeId;
        existing.MatchDate = match.MatchDate;
        existing.Venue = match.Venue;
        existing.Matchday = match.Matchday;

        _logger.LogInformation("Updating match with ID: {MatchId}", id);
        await _matchRepository.UpdateAsync(existing);
    }

    public async Task DeleteAsync(int id)
    {
        var existing = await _matchRepository.GetByIdAsync(id);
        if (existing == null)
            throw new KeyNotFoundException($"No se encontró el partido con ID {id}");

        if (existing.Status != MatchStatus.Scheduled)
            throw new InvalidOperationException(
                "Solo se pueden eliminar partidos con estado Scheduled");
    }

```

```

        _logger.LogInformation("Deleting match with ID: {MatchId}", id);
        await _matchRepository.DeleteAsync(id);
    }

    public async Task UpdateStatusAsync(int id, MatchStatus newStatus)
    {
        var match = await _matchRepository.GetByIdAsync(id);
        if (match == null)
            throw new KeyNotFoundException($"No se encontró el partido con ID {id}");

        var validTransition = (match.Status, newStatus) switch
        {
            (MatchStatus.Scheduled, MatchStatus.InProgress) => true,
            (MatchStatus.InProgress, MatchStatus.Finished) => true,
            (MatchStatus.Scheduled, MatchStatus.Suspended) => true,
            (MatchStatus.InProgress, MatchStatus.Suspended) => true,
            _ => false
        };

        if (!validTransition)
            throw new InvalidOperationException(
                $"No se puede cambiar de {match.Status} a {newStatus}");

        match.Status = newStatus;

        _logger.LogInformation(
            "Updating match {MatchId} status to {NewStatus}", id, newStatus);
        await _matchRepository.UpdateAsync(match);
    }
}

```

5 validaciones en CreateAsync

Este Service es el más robusto del proyecto. Al crear un partido valida: (1) torneo existe y está InProgress, (2) equipos son diferentes, (3) ambos equipos existen, (4) ambos equipos están inscritos en el torneo, (5) árbitro existe. Cada validación falla con un mensaje descriptivo.

10. Crear los DTOs de Match

SportsLeague.API/DTOs/Request/MatchRequestDTO.cs

```

namespace SportsLeague.API.DTOs.Request;

public class MatchRequestDTO
{
    public int TournamentId { get; set; }
}

```

```

    public int HomeTeamId { get; set; }
    public int AwayTeamId { get; set; }
    public int RefereeId { get; set; }
    public DateTime MatchDate { get; set; }
    public string Venue { get; set; } = string.Empty;
    public int Matchday { get; set; }
}

```

SportsLeague.API/DTOs/Response/MatchResponseDTO.cs

```

using SportsLeague.Domain.Enums;

namespace SportsLeague.API.DTOs.Response;

public class MatchResponseDTO
{
    public int Id { get; set; }
    public int TournamentId { get; set; }
    public string TournamentName { get; set; } = string.Empty;
    public int HomeTeamId { get; set; }
    public string HomeTeamName { get; set; } = string.Empty;
    public int AwayTeamId { get; set; }
    public string AwayTeamName { get; set; } = string.Empty;
    public int RefereeId { get; set; }
    public string RefereeFullName { get; set; } = string.Empty;
    public DateTime MatchDate { get; set; }
    public string Venue { get; set; } = string.Empty;
    public int Matchday { get; set; }
    public MatchStatus Status { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime? UpdatedAt { get; set; }
}

```

SportsLeague.API/DTOs/Request/UpdateMatchStatusDTO.cs

```

using SportsLeague.Domain.Enums;

namespace SportsLeague.API.DTOs.Request;

public class UpdateMatchStatusDTO
{
    public MatchStatus Status { get; set; }
}

```

11. Actualizar AutoMapper

Agregar al MappingProfile.cs

```

// Match mappings

```

```

CreateMap<MatchRequestDTO, Match>();
CreateMap<Match, MatchResponseDTO>()
    .ForMember(dest => dest.TournamentName,
        opt => opt.MapFrom(src => src.Tournament.Name))
    .ForMember(dest => dest.HomeTeamName,
        opt => opt.MapFrom(src => src.HomeTeam.Name))
    .ForMember(dest => dest.AwayTeamName,
        opt => opt.MapFrom(src => src.AwayTeam.Name))
    .ForMember(dest => dest.RefereeFullName,
        opt => opt.MapFrom(src =>
            src.Referee.FirstName + " " + src.Referee.LastName));

```

💡 Mapeo de múltiples Navigation Properties

Usamos 4 ForMember para mapear datos de las 4 Navigation Properties a campos planos en el DTO. El frontend recibe TournamentName, HomeTeamName, AwayTeamName y RefereeFullName directamente sin necesidad de hacer llamadas adicionales.

12. Crear el MatchController

SportsLeague.API/Controllers/MatchController.cs

```

using AutoMapper;
using Microsoft.AspNetCore.Mvc;
using SportsLeague.API.DTOs.Request;
using SportsLeague.API.DTOs.Response;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.API.Controllers;

[ApiController]
[Route("api/[controller]")]
public class MatchController : ControllerBase
{
    private readonly IMatchService _matchService;
    private readonly IMapper _mapper;

    public MatchController(
        IMatchService matchService,
        IMapper mapper)
    {
        _matchService = matchService;
        _mapper = mapper;
    }

    [HttpGet("tournament/{tournamentId}")]
    public async Task<ActionResult<IEnumerable<MatchResponseDTO>>> GetByTournament(
        int tournamentId)
    {

```

```

        try
        {
            var matches = await _matchService.GetAllByTournamentAsync(tournamentId);
            return Ok(_mapper.Map<IEnumerable<MatchResponseDTO>>(matches));
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<MatchResponseDTO>> GetById(int id)
    {
        var match = await _matchService.GetByIdAsync(id);
        if (match == null)
            return NotFound(new { message = $"Partido con ID {id} no encontrado" });
        return Ok(_mapper.Map<MatchResponseDTO>(match));
    }

    [HttpPost]
    public async Task<ActionResult<MatchResponseDTO>> Create(MatchRequestDTO dto)
    {
        try
        {
            var match = _mapper.Map<Match>(dto);
            var created = await _matchService.CreateAsync(match);
            var matchWithDetails = await _matchService.GetByIdAsync(created.Id);
            var responseDto = _mapper.Map<MatchResponseDTO>(matchWithDetails);
            return CreatedAtAction(nameof(GetById), new { id = responseDto.Id },
responseDto);
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
        catch (InvalidOperationException ex)
        {
            return Conflict(new { message = ex.Message });
        }
    }

    [HttpPut("{id}")]
    public async Task<ActionResult> Update(int id, MatchRequestDTO dto)
    {
        try
        {
            var match = _mapper.Map<Match>(dto);
            await _matchService.UpdateAsync(id, match);
            return NoContent();
        }
        catch (KeyNotFoundException ex) { return NotFound(new { message = ex.Message
}); }
        catch (InvalidOperationException ex) { return Conflict(new { message =
ex.Message }); }
    }

```

```

[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id)
{
    try
    {
        await _matchService.DeleteAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException ex) { return NotFound(new { message = ex.Message }); }
    catch (InvalidOperationException ex) { return Conflict(new { message = ex.Message }); }
}

[HttpPatch("{id}/status")]
public async Task<ActionResult> UpdateStatus(int id, UpdateMatchStatusDTO dto)
{
    try
    {
        await _matchService.UpdateStatusAsync(id, dto.Status);
        return NoContent();
    }
    catch (KeyNotFoundException ex) { return NotFound(new { message = ex.Message }); }
    catch (InvalidOperationException ex) { return Conflict(new { message = ex.Message }); }
}

```

13. Actualizar Program.cs

Agregar los nuevos registros

```

// — Repositories (agregar) —
builder.Services.AddScoped<IMatchRepository, MatchRepository>();

// — Services (agregar) —
builder.Services.AddScoped<IMatchService, MatchService>();

```

14. Crear y Aplicar la Migración

```

dotnet ef migrations add AddMatchEntity \
  --project SportsLeague.DataAccess \
  --startup-project SportsLeague.API

```

```
dotnet ef database update \
  --project SportsLeague.DataAccess \
  --startup-project SportsLeague.API
```

💡 Verificar la migración

Revisar que el archivo generado tenga: CreateTable para Matches con 4 FKs, índices en cada FK, OnDelete Cascade solo en TournamentId, y OnDelete Restrict en HomeTeamId, AwayTeamId y RefereeId.

15. Probar la API

Para probar Match necesitamos datos previos. Seguir estos pasos en orden:

Paso 1: Crear datos base (si no existen)

1. POST /api/Team → Crear 2 equipos (ej: Nacional, Medellín)
2. POST /api/Referee → Crear 1 árbitro
3. POST /api/Tournament → Crear 1 torneo
4. POST /api/Tournament/1/teams → Inscribir equipo 1 y equipo 2
5. PATCH /api/Tournament/1/status → { "status": 1 } (InProgress)

Paso 2: Crear un partido

```
POST /api/Match
{
  "tournamentId": 1,
  "homeTeamId": 1,
  "awayTeamId": 2,
  "refereeId": 1,
  "matchDate": "2025-03-15T15:00:00",
  "venue": "Atanasio Girardot",
  "matchday": 1
}
```

Verificaciones esperadas

Acción	Código	Resultado
POST /api/Match	201	Partido creado con nombres de equipos y árbitro
POST con torneo Pending	409	Solo se programan en InProgress

Acción	Código	Resultado
POST con HomeTeam == AwayTeam	409	Equipos deben ser diferentes
POST equipo no inscrito en torneo	409	Equipo no inscrito
POST con árbitro inexistente	404	Árbitro no encontrado
GET /api/Match/tournament/1	200	Lista de partidos con detalles
GET /api/Match/1	200	Partido con todos los nombres
PATCH Scheduled → InProgress	204	Transición válida
PATCH InProgress → Finished	204	Transición válida
PATCH Finished → Scheduled	409	Transición NO válida
PATCH Scheduled → Suspended	204	Transición válida
DELETE partido InProgress	409	Solo eliminar Scheduled
DELETE equipo con partidos	500	Restrict impide eliminación

16. Resumen de Conceptos Aprendidos

Concepto	Aplicación en esta Fase
Múltiples FK a misma tabla	HomeTeamId y AwayTeamId ambos apuntan a Team
DeleteBehavior.Restrict	Evita ciclos de cascada en Team y Referee
DeleteBehavior.Cascade	Solo en Tournament (eliminar torneo elimina partidos)
4 Navigation Properties	Tournament, HomeTeam, AwayTeam, Referee en Match
2 colecciones en Team	HomeMatches y AwayMatches (una por cada FK)
Múltiples .Include()	4 JOINS en una query para cargar todos los detalles
Máquina de estados	MatchStatus con transiciones incluyendo Suspended
5 validaciones en Create	Torneo, equipos diferentes, inscritos, árbitro
Service con 5 repositorios	MatchService coordina la validación más compleja
ForMember múltiple	4 mappings de Navigation Properties en AutoMapper

17. Próxima Fase

En la Fase 5 agregaremos:

- Entity MatchResult con relación 1:1 a Match

- Entity Goal con FK a Match y Player, enum GoalType
- Entity Card con FK a Match y Player, enum CardType
- Validaciones: solo registrar goles/tarjetas en partidos InProgress o Finished
- Validación: el jugador debe pertenecer a uno de los dos equipos del partido
- Validación del minuto del gol/tarjeta (rango válido)